Dependency-Based Task Scheduler Using Topological Sorting

Final Project Report, Algorithms (Section 001), Fall 2026

American University of Afghanistan. Instructor: Dr. Umut Tosun. Submitted: 5 November 2026

Member	AUAF ID	Role	Main contribution
Raufa Rauf	49277	Graph Architect	`TaskGraph` class and the input parser; sample input files
Sweeta Latifi	45884	Algorithm Developer	Kahn's algorithm, first working version of the program, Tkinter GUI
Reza Panahi	49780	Cycle Checker & Tester	Cycle reporting, test cases, live demo and presentation
Ehsanullah Naseri	49783	Interface & I/O Lead	Command-line input/output, code revisions, README and this report

Table 1. Team members and roles

1. Problem and approach

Many jobs cannot start until others are finished: a student cannot take Algorithms before Data Structures, and an app should not be deployed before it has been tested. We model each task as a vertex of a directed graph and each rule "X must be done before Y" as an edge X → Y. A valid schedule is an ordering of the vertices in which every edge points forward, which is exactly a topological order. Such an order exists if and only if the graph has no directed cycle [2], so the program has two jobs: produce an order when one exists, and explain the problem when it does not.

2. Program design

The program is written in Python 3 with only the standard library. Its parts follow the four roles in the assignment, so each could be built and tested separately.

Graph and parser. `TaskGraph` stores the graph as an adjacency list (a dictionary mapping each task to the tasks that depend on it) and keeps tasks in the order they were entered, so the output is the same on every run. `parse_lines` reads the "Tasks:" line and one "X -> Y" rule per line, skips comments and blank lines, ignores repeated edges, and adds any task that appears only in a rule. A line it cannot read raises an error that names the line number.

Input and output. Sorting and cycle checking are covered in Section 3. The command-line program takes an input file or interactive typing, can save the result with `-o`, and returns exit code 0, 1 or 2 for a valid order, a cycle and an input error. The GUI (`task_scheduler_gui.py`, Section 5) imports the same functions, so the two interfaces always give the same answer.

3. Algorithm

We used Kahn's algorithm [1]. It first counts the in-degree of every task, that is, how many prerequisites it is still waiting for. All tasks with in-degree 0 go into a FIFO queue. The loop then removes a task from the queue, appends it to the order and lowers the in-degree of each task that depends on it; a task whose count reaches 0 joins the queue. On the assignment sample only A starts at 0. Removing A frees B, B frees C and C frees D, which gives A → B → C → D.

We preferred it to the DFS version for three reasons. Cycle detection comes almost for free: if the queue empties while some tasks still have in-degree above 0, those tasks lie on a cycle or wait for one. The queue is filled in input order, so the result is predictable and easy to check by hand. And the loop is iterative, while a recursive DFS would hit Python's default recursion limit of 1000 on a long chain.

Kahn's algorithm shows that a cycle exists but not where it is. To give the user something to fix, `find_cycle` runs an iterative depth-first search with white/gray/black colouring over the leftover tasks only. The first edge that reaches a gray (still open) vertex is a back edge, and following parent links from it gives the cycle, which is printed under the required error line, e.g. "Cycle: B -> C -> D -> B". Leftover tasks that only depend on a cycle, such as E in S → A → B → C → A, C → E, are not reported as part of it.

Complexity. Counting in-degrees visits each edge once. Every task enters and leaves the queue at most once, and each edge is examined once, when its source task is removed. The running time is therefore O(V + E), and the cycle search adds at most another O(V + E). Memory use is O(V + E) for the adjacency list, the in-degree table and the queue.

4. Testing and results

Table 2 lists the sample files in the `samples` folder. Each has a matching `outputN.txt` produced by the program.

Input file	What it checks	Program output
`input1_valid.txt`	Assignment sample, simple chain	`Valid Task Order: A -> B -> C -> D`
`input2_cycle.txt`	D -> B closes a loop	`Error: Cycle detected. No valid task order.` `Cycle: B -> C -> D -> B`
`input3_courses.txt`	7 courses, two starting points	`Valid Task Order: Intro -> Math -> DataStructures -> Algorithms -> Databases -> AI -> Capstone`
`input4_self_loop.txt`	A task that depends on itself	`Error: Cycle detected. No valid task order.` `Cycle: Build -> Build`
`input5_app_release.txt`	Parallel branches, a task not listed on the Tasks line	`Valid Task Order: Requirements -> Write user guide -> Database design -> Mobile UI -> API -> Translate UI -> Testing -> Deploy`
`input6_bad_format.txt`	Wrong arrow symbol (=>)	`Input error: Line 4: cannot parse 'A => B'`

Table 2. Sample inputs and outputs

The file `tests/test_task_scheduler.py` contains 27 automated test cases written with `unittest`. They cover the parser (comments, repeated tasks and edges, names with spaces, five kinds of malformed line), ordering (the sample, reversed rules, disconnected tasks) and cycles (two tasks, a self-dependency, a cycle with tasks before and after it). Randomised tests add 200 acyclic graphs, where every edge must point forward in the output, and 200 graphs with one back edge, where each step of the reported cycle must be a real edge. A chain of 100,000 tasks confirms there is no recursion problem, and the exit codes are checked too. All 27 tests pass in under a second. To test the tests, we inserted two deliberate bugs (in the cycle search and in the in-degree update); the suite failed both times.

5. Graphical interface

Figure 1 shows the GUI. The user types tasks and rules or loads a sample file, then presses Find Order (or Ctrl+Enter). For a valid order the tasks are drawn left to right in columns, where a task's column is the length of the longest chain of prerequisites before it, and the number under each task is its position in the order. When a cycle is found the tasks are placed on a circle and the cycle is drawn in red. The result can be saved to a text file.

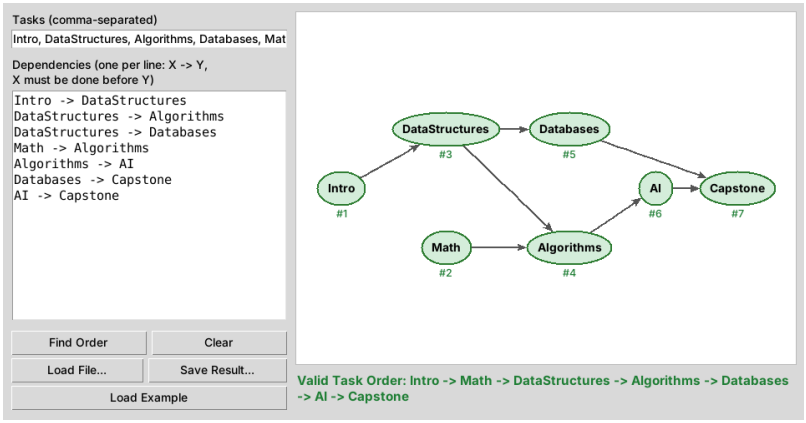


Figure 1. The GUI after loading input3_courses.txt (course prerequisites)

Revising the first version, we fixed three problems found in testing: long task names spilled outside their circles, an arrow could pass straight through an unrelated task, and long results were cut off at the window edge. Nodes now grow with their names, such arrows are curved around the task, and the result wraps. We also added three sample files, the tests and a README.

6. Limitations and possible extensions

When several valid orders exist, the program reports one of them; in the course example Math could just as well come before Intro. When the input contains more than one cycle only one is shown, so fixing it may reveal the next. Possible extensions are a priority queue to break ties (for example by task duration), grouping tasks into stages that can run in parallel, and critical-path analysis once tasks have durations.

References

[1] A. B. Kahn, "Topological sorting of large networks," *Communications of the ACM*, vol. 5, no. 11, pp. 558–562, 1962.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009, Sec. 22.4.